Telemetry Data-Pipeline Mini-Assessment

24-hour take-home capability assessment

Candidate	Mbaye Ndiaye Diouf
Track	Development / Automation & AI
Released	**Tuesday, June 16, 2026 — 10:00 AM EDT**
Due	**Wednesday, June 17, 2026 — 10:00 AM EDT (24 hours)**

Purpose

This exercise assesses how you build a small but solid data-processing pipeline: ingesting a stream of records, validating and cleaning messy real-world data, computing useful results, and producing structured output and alerts. This is the kind of automation work on our development track, and it is close to the telemetry and data-pipeline work you have already done. We value a smaller pipeline that works correctly and is well-explained over a large one that is broken or unvalidated.

The challenge

Build a tool (command-line, or a lightweight service with a small dashboard if you prefer) that ingests a stream of device telemetry readings, processes them, and reports on the health of the devices. Picture a fleet of sensors reporting periodic readings — temperature, battery level, signal strength, and a timestamp — where some readings arrive malformed, out of order, duplicated, or missing fields. Your pipeline must handle that gracefully and turn the raw stream into clean, structured results plus alerts when something looks wrong.

The input data — and how to stay self-contained

You generate your own sample data; nothing is provided by us. Create a dataset of a few hundred telemetry records (a JSON or CSV file, or a small generator script) for a handful of simulated devices. Each record should include at least a device ID, a timestamp, and a few numeric fields (for example temperature, battery percentage, signal strength).

Deliberately make the data realistic and messy, and document what you injected, so we can see your pipeline handle it. For example: a few records with missing or null fields, some out-of-range or impossible values, a couple of duplicate records, some out-of-order timestamps, and at least one malformed/garbage line.

> We are assessing how your pipeline handles imperfect data, so messy input is the point. If you prefer, you may stream records in over time (for example, reading line-by-line, or via a simple queue) rather than loading them all at once — but a straightforward batch read is also fully acceptable.

Required behavior

1. **Ingest —** Read the telemetry records from your source (file, generator, or stream — your choice; document it).
2. **Validate & clean —** Validate and clean each record: drop or quarantine malformed lines, handle missing fields, flag out-of-range values, and de-duplicate. Bad records must not crash the run — they should be counted and set aside.
3. **Aggregate —** Per device, compute useful aggregates over the clean data — for example min / max / average temperature, latest battery level, reading count, and the time span covered.
4. **Alerting —** Generate alerts for conditions that matter — for example battery below a threshold, a temperature out of a safe range, or a device that appears to have stopped reporting (a gap in its timestamps).
5. **Structured output & summary —** Write a single structured output file with the per-device summary and the alerts (JSON or CSV), and print a short run summary: records read, accepted, rejected (by reason), and number of alerts.

README — design decisions (read closely)

Include a README that covers:

- **Run instructions:** How to install and run the tool, and how to (re)generate or point it at your sample data.
- **Data quality:** What messy conditions you injected into the data and how your pipeline handles each one.
- **Pipeline design:** Your processing design — how records flow through ingest, validation, aggregation, and alerting, and why you structured it that way.
- **Trade-offs & next steps:** Trade-offs given the time limit and what you would build next (true streaming, persistence, scaling to many devices, a real dashboard).

Stretch goals (optional, not required)

- **Tests:** Automated tests, including a test that feeds the validator deliberately malformed records.
- **Visualization:** A small dashboard or chart of a device's readings over time.
- **Streaming:** True streaming/real-time processing rather than a single batch pass.
- **Persistence / polish:** Persistence to a small database, or a clean, considered interface (CLI flags or minimal web UI).

How we evaluate (100 points)

25 **Working software —** the pipeline runs as documented and produces correct per-device summaries and alerts.

25 **Data validation & robustness —** messy input is handled cleanly — bad records are caught and counted, not crashed on or silently mis-processed.

20 **Code quality & structure —** readable, organized code with clean separation between ingest, validation, aggregation, and output.

15 **Communication —** clear README and walkthrough; data-quality handling and design well explained.

15 **Stretch / initiative —** tests, visualization, streaming, or thoughtful extras beyond the minimum.

Ground rules

- **Your own work:** This is an individual assessment. The work must be your own.
- **AI tools allowed, with disclosure:** You may use AI coding assistants, documentation, and online references — that reflects how we actually work. If you use an AI assistant, add a short note in your README saying which one and for what. Using AI is not penalized; being unable to explain your own submission is.
- **Fully self-contained:** Everything you need is in this brief. You do not need anything from Praxtion — no accounts, no credentials, no data, no environment. Do not request access to any Praxtion system.
- **No real data:** Do not use any real client or personal data. Use only the simulated telemetry data you generate.
- **Questions:** If something is genuinely ambiguous, make a reasonable assumption, state it in your README, and proceed. Handling ambiguity well is part of what we assess. You may email one consolidated question to the address below, but a same-day reply is not guaranteed, so do not block on it.
- **Done beats big:** We would rather see a smaller, working, well-explained submission than a large, broken, undocumented one. Scope to what you can finish and do well.

How to submit

Before 9:00 AM EST on Tuesday, June 16, 2026, reply to info@praxtion.com (replying to the assessment email is fine) and include:

1. **Repository link —** A link to a public (or access-shared) GitHub repository containing your code, your README, your sample-data file or generator, and an example output file.
2. **Walkthrough —** A short walkthrough — either a written section in your README or a 3–5 minute screen recording (for example, Loom) — covering what you built, how you handle messy data, what you would do next, and any assumptions you made.
3. **Effort & tools note —** A note of roughly how many hours you spent and which AI tools, if any, you used.

Submitting earlier than the deadline is welcome and does not count against you. If you run out of time, submit what you have with a short note on what remains — partial, honest, well-explained work is evaluated fairly.

Questions during the window: email info@praxtion.com (one consolidated message; same-day reply not guaranteed).